

RingGraph: project readout

This document explains the project, the work, and the results. It uses simple English (ASD-STE100 style): short sentences, one meaning per word. Numbers come from the files in `reports/`. Date of the numbers: 7 October 2026.

1. The project in one minute

- The project tests one claim: **graph structure plus delayed fraud knowledge finds more fraud than a plain tabular model**.
 - A plain tabular model looks at one payment at a time. We call it **M1**.
 - A **graph** links clients that share a device or a browser. A group of linked clients is a **ring**.
 - **Delayed fraud knowledge** is fraud that the bank already knew about. A fraud label is known only some days after the payment (the chargeback delay).
 - We use a fraud label only after that delay. This rule stops future information from leaking into the model.
 - Result: the graph model finds **31% more fraud than M1** at the same false-alarm rate (30-day delay).
 - Most of this gain comes from the **client's own history**, not from links between clients. Links add a smaller gain.
 - The project also builds a **catalogue of 550 candidate rings** and a web page to look at them.
-

2. The data

- Data set: IEEE-CIS Fraud Detection (Kaggle).
- 590,540 payments. 3.5% are fraud.
- Each payment has 436 columns after the identity table is joined.
- Identity columns (device, browser, screen) exist for only about a quarter of the payments.
- The data set has **no client ID**. We rebuild a client from three values: `card1`, `addr1`, and the day the card began ($D1n = \text{day} - D1$).
- Result: 217,850 clients. 57% of them have only one payment.
- The data is not in the repository. A script downloads it with a Kaggle account.

The fixed time split

Part	Days	Use
Train	0 to 119	The model learns here
Validation	120 to 150	We choose settings here
Test	151 to 181	We report results here. We never tune on it.

- The split boundaries are constants in the code. Nobody can change them with a setting.

- The test part has 89,326 payments and 3,114 frauds.

3. The rules that keep the test fair

- **Rule 1.** A label can enter a feature for a payment at time t only if the fraud was confirmed at or before t minus the delay.
- **Rule 2.** A graph feature for a payment at time t uses only payments before t .
- **Rule 3.** We never tune on the test part.
- **Rule 4.** M1 is never changed. We do not spend time to make it stronger.
- **Rule 5.** We always show three models together: M1, the control, and the graph model. This shows how much of the gain is really the graph.
- **Rule 6.** Control and graph models use the same tuning search as M1. The same search space, the same number of trials, the same goal.
- **Rule 7.** We repeat each test with several random seeds. We use a paired bootstrap to get the confidence interval of the **difference** between two models.
- 83 automatic tests check these rules. All 83 tests pass. Several tests are leakage tests. They fail if a rule is broken.

4. The three models

Name	Inputs	Purpose
normal (M1)	431 tabular features. One payment at a time.	The baseline. Unchanged.
control	M1 + delayed fraud history on tabular keys (card, address, email, client)	Shows what a tabular team can add without a graph
graph	control + shared-entity exposure, ring breadth and recency, neighbourhood behaviour, two-hop spread, client profile	The model we test

- All models use LightGBM (gradient-boosted trees).
- The main metric is **TPR at 1% FPR**: the share of frauds caught when only 1% of good payments raise an alarm. We also report 0.1% FPR.
- ROC-AUC and PR-AUC are secondary metrics.

5. Two settings, always reported apart

1. **Causal (deployable).** At time t the model sees only earlier payments and labels confirmed by t . This is what a bank could run. We test delays of 7, 14, 30, 60 and 90 days. **30 days is the realistic headline.**
2. **Offline (retrospective, Kaggle style).** The model also sees whole-data client totals that include future payments. It never sees test labels. We add features step by step (levels L0 to L4).

6. Results

6.1 Causal setting, test part, TPR at 1% FPR

* means the 95% interval of the difference excludes zero.

Delay	Seeds	M1	Control	Graph	Graph – M1	Graph – control	At 0.1% FPR: M1 → graph
7 days	3	0.426	0.619	0.636	+0.192* (+49%)	+0.016*	0.233 → 0.375
14 days	5	0.426	0.584	0.603	+0.165* (+42%)	+0.023*	0.233 → 0.336
30 days	5	0.426	0.538	0.560	+0.120* (+31%)	+0.027*	0.233 → 0.310
60 days	3	0.426	0.501	0.509	+0.065* (+19%)	+0.005	0.233 → 0.268
90 days	3	0.426	0.472	0.478	+0.040* (+12%)	+0.007	0.233 → 0.250

- Graph beats M1 at every delay. The gain is largest for short delays.
- A short delay lets more confirmed fraud reach the features.
- The graph beats the control at 7, 14 and 30 days. It does not beat the control at 60 and 90 days.
- At 30 days and 0.1% FPR, graph minus control is –0.018. This is not significant. We do not claim a graph gain at 0.1% FPR.

6.2 What the 30-day numbers mean in payments

- The test part has 3,114 frauds.
- At 1% false-alarm rate, M1 catches 1,327 frauds.
- The graph model catches 1,743 frauds. That is 416 more.
- At 14 days, the graph model catches 1,877 frauds, with precision 0.69 (M1: 0.61).

6.3 Offline setting (30-day delay, 5 seeds)

Level	What is added	TPR at 1% FPR	TPR at 0.1% FPR
L0	M1	0.426	0.233
L1	whole-data client totals	0.500*	0.254
L2	own-client delayed history	0.547*	0.317*
L3	graph features	0.556*	0.331
L4	client smoothing	0.556	0.331

- Smoothing (average or maximum over the client) adds nothing. Validation chose the setting. The test difference L4 – L3 is –0.0006.

6.4 Where the gain comes from

- **One feature family added to M1 (30 days, 1% FPR):**
- neighbourhood behaviour with no labels: 0.434

- client profile: 0.448
- delayed exposure over many keys: 0.553
- delayed exposure plus two-hop spread: 0.552
- control: 0.538. Graph: 0.560.
- **SHAP check.** SHAP shows which features push a score. For the 483 frauds that only the graph model catches:
- delayed exposure over many keys: 35%
- M1 tabular features: 34%
- client profile: 16%
- own history (tabular keys): 8%
- neighbourhood behaviour: 4%
- device-graph labels: 2%
- two-hop spread: 1%
- The top features are all about the **same client's** confirmed-fraud record.

Honest conclusion. Most of the lift over M1 is the client's own confirmed-fraud history. Links between clients add a smaller gain (+0.016 to +0.027 at 7 to 30 days).

6.5 Why we cannot get a much larger graph gain

- About 79% of fraud is the first fraud of a client. It has no link to any known fraud.
- For these frauds no key-sharing feature can help.
- For new clients (no earlier payments), no model beats M1 by a significant margin.

6.6 Earlier result: graph shape without labels

- Features that describe only the shape of the graph (degree, triangles, PageRank, community totals) gave **no lift**.
- Five seeds, full test part, TPR at 1% FPR: M1 0.425, M1 plus graph shape 0.419. Difference -0.006 . This is inside the noise.
- A single training run changes the score by about 0.033 between seeds. This is larger than any model difference in this test.
- The new gain comes from **labels with a delay**, not from graph shape alone.

7. Rings

- A ring is a group of 3 or more clients linked by shared device or browser attributes.
- The ring catalogue has **550 rings** and **4,762 clients**. The median ring has 5 clients. The largest has 165.
- The link rule: clients must share a device or browser attribute. A shared value may link at most 10 clients (hub limit). Hub values are too common to be useful.
- Ring rules were chosen by measurement. A ring set must not merge most clients into one big group. The largest group is 31% of linked clients.
- Rings are built **without labels**. Fraud labels are shown only for display.

Ring scores

- Each ring gets four scores: **density**, **synchrony**, **concentration**, **tightness**. Burst share is a fifth measure.
- Density: how many of the possible client-to-client links exist.
- Synchrony: how often clients transact at the same time, compared to a time-shifted random baseline.
- Concentration: clients per shared attribute value.
- Tightness: how equal the payment amounts are.
- Burst share: the share of a ring's payments inside one time window.
- Burst share is the best ranking: the top 50 rings hold 1.69 times the fraud clients that ring size alone predicts. The four-score composite reaches 1.27.

Fraud inside rings

Measure	Inside rings	Whole data	Ratio
Fraud share of clients	19.3%	3.7%	5.2 times
Fraud share of payments	9.1%	3.5%	2.6 times

- 112 of 550 rings hold 2 or more fraud clients. These 112 rings hold 837 of the 917 fraud clients in rings.
- 358 rings hold no fraud. 12 rings are entirely fraud (3 to 9 clients each).
- The top ring (ring 263) has 5 clients, one device fingerprint, 5 payments of \$25 within 2 minutes. All 5 clients are fraud.
- Rings are built over the whole period. They are a retrospective view, not a tested live detector.
- Safe wording: *"The graph groups clients into candidate rings. Clients in them are about 5 times more likely to be fraud than the base rate."*

Checks on the ring work

- **Trap A check.** Fraud labels in this data set follow the client, not the payment. 96.6% of clients with 2 or more payments are all-fraud or all-good, against 85.1% by chance. For this reason we study only links **between** clients.
- **Shuffle test.** If we shuffle the labels, the ring finding falls from 1.45 to 0.97 (no signal). This shows that the finding is not a software error.
- **Link band.** With a hub limit of 50, card1 groups hold fewer fraud clients than chance (-17.5 sd). With a limit of 10 they hold more than chance ($+10.6$ sd). The limit decides the sign. This is why hubs are cut off.
- **Coordination.** In the smallest size band, ring components show 189 same-time bursts against a baseline of 1.25 (151 times). Fraud-free components of the same size show 46 times.

8. The software

8.1 Python library (`src/fds/`)

- **Data:** `ingest` joins the two CSV files. `schema` lists columns and a deny list of columns that a model must never use. `splits` holds the time split. `paths` and `config` set file places and run settings.
- **Clients and links:** `entities` rebuilds the client ID. `links` makes client-to-client links. `snapshots` builds the graph for each past time window. `synchrony` tests for coordination.
- **Features:** `label_features` (delayed fraud history), `relational_features` (shared-entity exposure, neighbourhood, two-hop), `client_profile`, `offline_features`, `structural` and `community_features` (graph shape), `risk_propagation` (tested, no gain, not used).
- **Models and metrics:** `models` trains LightGBM. `tuning` runs the hyperparameter search. `evaluation` computes TPR at fixed FPR and the paired bootstrap.
- **Rings:** `rings` finds rings and scores them. `communities` and `graph_load` / `graphdb` (Neo4j) are optional.
- **Support:** `artifacts` writes files with a manifest and a stale-file guard. `rng` makes fixed seeds. `cli` is the shared start-up for scripts.

8.2 Scripts (`scripts/NN_name.py` , run in number order)

- 00 to 20: download, join, profile the data.
- 30 to 50: rebuild clients, choose the hub band, run the three gates (label clustering, link signal, coordination).
- 60 to 63: optional Neo4j graph and picture.
- 70 to 71: train and tune M1.
- 80 to 82: graph-shape features and M2.
- 83 to 87: delayed-label, relational, profile, and offline feature tables.
- 90 to 92: older comparison scripts.
- 93: causal sweep (headline). 94: tune control and graph. 96: offline ladder. 98: summary table and `headline.json`.
- 95: build the ring catalogue. 99: fraud rate inside rings. 100: SHAP.

8.3 Tests (`tests/`)

- 83 tests. They check splits, client rebuild, the delayed-label rule, graph-time rule (Trap B), offline features, synchrony, metrics, config, and the stale-file guard.

8.4 API (`api/main.py` , FastAPI)

- It reads saved files. It does not need a database.
- `/health`, `/rings`, `/rings/{id}` (detail, `/subgraph`, `/events`).
- `/metrics/models` (model scores and delayed-label results), `/metrics/summary` (M1 numbers and ring totals), `/metrics/shap`, `/metrics/axes`, `/metrics/sweep`.
- The ring list can sort by outlier score (most extreme single score).

8.5 Web page (web/, React, TypeScript, Vite)

- **Overview page.** Live number strip, ring score explainer, four findings, an eight-step pipeline, the top rings.
- **Explore page.** Ring list with sort buttons (including Outlier), a graph picture of the selected ring, and an evidence panel with a time strip. The panel states how many clients transacted close together.
- **Results page.** Model scores, delayed-label results with a delay selector, where-the-gain-comes-from charts (feature families and SHAP), ring evidence (5.2 times figure and a scatter of all rings), ring ranking, and a threshold slider for M1.
- Numbers on the pages come from the API. Pipeline step numbers come from the committed reports.

8.6 What was fixed while we built the pages

- Three score formulas in the explainer did not match the code. They now match.
- Two old pipeline numbers came from an older link band. The page now states both.
- Labels such as "linked clients" now say "clients in rings".

9. How to run it

1. Create the environment: `python3.11 -m venv .venv`, then install `requirements.txt`. Install packages only in `.venv`.
2. Put the Kaggle key in `~/.kaggle/kaggle.json` and accept the competition rules.
3. Run the scripts in number order (see `README.md`).
4. Run the demo without data: committed files in `data/rings`, `reports`, and `preds` feed the API.
5. Start the API: `uvicorn api.main:app --port 8000`. Start the page: `npm run dev --prefix web`. Open `http://localhost:5173`.

10. Limits (state these before a question is asked)

- The control (own history) holds most of the gain. The graph adds a small gain.
- The gain falls as the delay grows. At 60 and 90 days graph does not beat the control.
- The 7, 60 and 90 day tests use 3 seeds. The 14 and 30 day tests use 5 seeds.
- The control and graph models were tuned once at 30 days and reused at other delays.
- M1 reads 44.2% in the committed run and 42.6% as the 5-seed mean. Both are correct. They are different measures.
- The offline setting uses future rows. It is not deployable. It is reported apart.
- Rings are retrospective. 358 of 550 rings have no fraud.
- About 79% of fraud has no link to earlier fraud. No linking feature can find it.
- We did not train a graph neural network. We expect little gain from it.
- We did not use the Kaggle test files. They have no labels.

11. Two-minute explanation

1. "We tried to find fraud using links between clients."
 2. "A plain model catches 42.6% of fraud when 1% of good payments raise an alarm."
 3. "We add fraud that the bank already knew about, but only after the chargeback delay. This keeps the test fair."
 4. "The new model catches 56.0%. That is 31% more."
 5. "Most of the gain comes from the client's own past fraud. Links between clients add a smaller gain of 2.7 points."
 6. "We also found 550 candidate rings. Clients in them are 5 times more likely to be fraud."
 7. "A model of graph shape alone gave no gain. We report that result too."
-

12. Words used

Word	Meaning
Fraud ring	A group of linked clients that act together
TPR	True positive rate: share of frauds caught
FPR	False positive rate: share of good payments that raise an alarm
Chargeback delay	Days between a payment and the confirmed fraud label
Leakage	Future information that enters a model by error
Seed	A number that fixes random choices. Different seeds give different training runs.
Paired bootstrap	A test that gives a confidence interval for the difference between two models, on the same samples
SHAP	A method that shows how much each feature pushes a score
Hub	An attribute value shared by very many clients
Control	M1 plus the client's own delayed history